1 Întrebare

Complet

Marcat 1,00 din 1,00

Întrebare cu flag

Se dă un graf neorientat conex reprezentând rețeaua de socializare Facebook (muchie x-y înseamnă *x* este prieten cu *y*). Gigel a făcut o postare pe Wall, care este văzută de prieteni. Se întreabă dacă postarea poate, ca prin mai multe share-uri, să fie văzută de Botzgorel (care nu este prieten cu Gigel). Descrieți un algoritm eficient.

Aplic algoritmul BFS pornind din nodul corespunzator lui Gigel. Daca nodul Botzgorel este accesibil din Gigel, atunci prin share-uri repetate postarea va putea fi vazuta de Botzgorel. (Graful fiind neorientat conex, raspunsul este da, acesta ar putea sa o vada.)

BFS(Gigel) / DFS(Gigel) și dacă vizităm nodul Botzgorel, atunci răspunsul este DA. Altfel NU.

2 întrebare

Complet

Marcat 0,50 din 1,00

Întrebare cu flag

Se dă un vector *v* cu *n* intervale de forma $[a_i, b_i]$. Se cere numărul maxim de intervale disjuncte (care nu se intersectează cu niciun interval). Descrieți un algoritm eficient.

Algoritmul este de tip greedy. Se sorteaza intervalele in ordine descrescatoare a capatului din dreapta. Se parcurg in ordine si se verifica pentru fiecare daca se intersecteaza cu ultimul inteval pastrat (initial aceasta multime fiind vida). Daca nu se intersecteaza, se adauga in multime si se updateaza ultimul interval pastrat. Altfel doar se continua parcurgerea. Cardinalul multimii dupa terminarea parcurgerii reprezinta raspunsul.

Greedy: se sortează apoi se parcurg intervalele de la stânga la dreapta. O(n log n).

3 Întrebare

Complet

Marcat 1,00 din 1,00

Întrebare cu flag

Un florar inginer s-a gândit să facă un program care să printeze pe ecran toate buchetele de câte *n* flori folosind maxim *m* tipuri de flori. Descrieți un algoritm pentru această problemă.

Algoritmul este de tip backtracking. Domeniul reprezinta multimea celor m tipuri de flori. La fiecare pas adaug pe rand fiecare element din domeniu si trec mai departe pentru fiecare element adaugat, incercand sa completez cu alte elemente. Cand sirul are n elemente, printez solutia si revin din recursivitate. Inlocuiesc ultimul element din solutia partiala cu urmatorul si continui.

Se foloseste backtracking pentru a genera toate buchetele de flori care respecta restrictiile din enunt.

4 întrebare

Complet

Marcat 0,50 din 1,00

Întrebare cu flag

Cum ați modifica algoritmul lui Dijkstra pentru a calcula al doilea cel mai scurt drum dintr-un graf?

Determin cel mai scurt drum folosind Dijkstra. Din acest drum, aleg pe rand cate o muchie si o elimin din graful initial, recalculand drumul minim aplicand Dijkstra. Dupa ce am eliminat si recalculat astfel pentru fiecare muchie, aleg cel mai scurt drum dintre noile drumuri. Acesta va fi al doilea cel mai scurt drum din graful initial. Complexitate V * E * log(V), pentru implementare cu heap binar.

1p - Se păstrează cele mai mici 2 distanțe posibile până la fiecare nod.

0.5p - se scoate pe rând câte o muchie și se rulează Dijkstra

5 Întrebare

Complet

Marcat 1,00 din 1,00

Întrebare cu flag

Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, dorește să determine toate mulțimile de fortărețe astfel încât chiar dacă una dintre fortărețele dintr-o mulțime ar fi bombardată, celelalte fortărețe din mulțime ar putea să comunice în continuare.

Punctele de articulatie se pot gasi folosind algoritmul lui Tarjan. Subgrafurile din graful initial care nu contin puncte de articulatie reprezinta solutia. (componente biconexe ale grafului initial)

Algoritmul adaptat presupune si retinerea unei stive cu muchiile de avansare si intoarcere parcurse pana la un moment dat, pentru a determina componentele biconexe.

Gasim toate componentele biconexe.

6 Întrebare

Complet

Marcat 1.00 din 1.00

Întrebare cu flag

Se dă un graf orientat cu *x* noduri, având capacitați numere din șirul Fibonacci. Se știe că există *A* surse și *B* noduri destinații. Descrieți un algoritm eficient pentru a transporta o cantitate maximă de flux de la toate sursele la toate destinațiile.

Avand surse si destinatii multiple, trebuie sa adaug o sursa unica cu arce de capacitate infinita spre sursele initiale si cu flux egal cu fluxul generat de acestea. Adaug si o destinatie unica si arce de capacitate infinita intre destinatiile initiale si aceasta, avand flux egal cu fluxul ce intra in destinatiile initiale. Pe graful nou format aplic algoritmul Edmonds-Karp pentru a determina fluxul maxim.

Se reunesc toate sursele într-o sursă fictivă S_0 (muchie de la S_0 la S_k de capacitate infinită) și toate destinațiile într-o destinație fictivă T_0 (muchie de la T_k la T_0 de capacitate infinită). S-a obținut o rețea de transport pe care se poate aplica Edmonds-Karp. Graful rezidual se obține ștergând nodurile si muchiile fictive adăugate. Complexitate O(n * m^2) sau O(MaxFlow * m). Se punctează cu 0.5p o soluție mai ineficienta.

Question 1

Complete

Mark 1.00 out of 1.00

Flag question

Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, se gândește să bombardeze o singură fortăreață, a.î. să întrerupă comunicarea între cât mai multe fortărețe și tabăra lui să câștige. Descrieți un algoritm pentru găsirea fortăreței.

Ma gandesc asemanator cu un graf cu retele, consider eficient sa se elimine nodul critic care ar sparge graful in cat mai multe componente conexe. Algoritmul pentru gasirea punctelor critice este Tarjan, folosind DFS si cautand proprietatea in care low_link[v]>=d[u]. Dupa aplicarea acestui algoritm vom afla toate nodurile critice, acum, in cazul in care sunt mai multe, avem de facut o alegere in functie de care nod sparge in cat mai multe componente conexe graful. Ca sa aflam asta as aplica backtracking, desi sunt sigur ca exista o solutie mai eficienta.

1p - Găsim toate punctele de articulatie și componentele biconexe din jurul lor. Gigel va bombarda punctul de articulație care are în jurul său cele mai multe componente biconexe.

0.5p - Întoarce orice punct de articulație

Comment:

Question **2**

Complete

Mark 1.00 out of 1.00

Flag question

Descrieți un algoritm pentru generarea tuturor grafurilor neorientate cu *n* noduri care NU sunt conexe.

As sparge 1-n in multe multimi, fiecare multime reprezinta o componenta conexa si dupa in interiorul multimii (presupunem are m elemente) voi avea m^2 solutii. In fiecare componenta ar trebui sa fac cu backtracking toate grafurile conexe posibile

Se folosește backtracking pentru a genera toate submulțimele de muchii care respectă restricțiile din enunț.

Comment:

Question **3**

Complete

Mark 1.00 out of 1.00

Flag question

Se dă un graf orientat cu *m* noduri, având capacitați numere pare pe arce. Se dau o sursă *S* și *p* noduri destinații. Descrieți un algoritm eficient pentru a transporta o cantitate maximă de flux de la sursă la toate destinațiile.

As incerca sa reduc problema la ceva cunoscut, adica sa am o singura destinatie si acolo sa aplic algoritmul Edmonds-Karp. Ca sa ajung la problema asta as conecta toate destinatiile cu edge-uri care suporta un flux infinit

Se reunesc toate sursele într-o sursă fictivă S_0 (muchie de la S_0 la S_k de capacitate infinită) și toate destinațiile într-o destinație fictivă T_0 (muchie de la T_k la T_0 de capacitate infinită). S-a obținut o rețea de transport pe care se poate aplica Edmonds-Karp. Graful rezidual se obține ștergând nodurile si muchiile fictive adăugate. Complexitate O(n * m^2) sau O(MaxFlow * m). Se punctează cu 0.5p o soluție mai ineficienta.

Comment:

Question **4**

Complete

Mark 1.00 out of 1.00

Flag question

Cum ați modifica algoritmul lui Dijkstra pentru a calcula al doilea cel mai scurt drum dintr-un graf?

In loc sa stochez pentru fiecare nod costul minim, voi stoca 2 costuri, costul minim si cel de al doilea cel mai mic. In PrioQueue voi avea grija sa stiu cand folosesc prima, respectiv a doua valoare (calea cea mai mica sau eventual a doua cea mai mica).

1p - Se păstrează cele mai mici 2 distanțe posibile până la fiecare nod.

0.5p - se scoate pe rând câte o muchie și se rulează Dijkstra

Comment:

Question **5**

Complete

Mark 1.00 out of 1.00

Flag question

Determinați dacă un graf este bipartit. Un graf este bipartit dacă mulțimea nodurilor se poate împărți în două submulțimi disjuncte, astfel încât să nu existe muchii între noduri din aceeași submulțime.

Colorez nodul sursa cu o culoare rosu de exemplu, toti vecinii ii colorez verde, apoi vecinii vecinilor ii fac rosii. Daca cand incerc sa colorez gasesc doi vecini cu aceeasi culoare inseamna ca nu e bipartit

1.

Se dă un graf orientat cu *m* noduri, având capacitați numere pare pe arce. Se dau o sursă *S* și *p* noduri destinații. Descrieți un algoritm eficient pentru a transporta o cantitate maximă de flux de la sursă la toate destinațiile.

R: Se reunesc toate sursele într-o sursă fictivă S_0 (muchie de la S_0 la S_k de capacitate infinită) și toate destinațiile într-o destinație fictivă T_0 (muchie de la T_k la T_0 de capacitate infinită). S-a obținut o rețea de transport pe care se poate aplica Edmonds-Karp. Graful rezidual se obține ștergând nodurile si muchiile fictive adăugate. Complexitate O(n * m^2) sau O(MaxFlow * m). Se punctează cu 0.5p o soluție mai ineficienta.

2.

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mici numere. Secvența este prea mare pentru a fi stocată integral în memorie.

Raspunsul meu: Folosim o structura de tipul maxHeap.
Citim cate un numar si daca in heapul nostru avem mai putin de k elemente, atunci il introducem in heap.

Daca citim un numar si heapul este deja plin(are k elemente), comparam noul numar cu elementul cel mai mare din heap (operatie ce se realizeaza in O(1)), iar daca noul element este mai mic inseamna ca putem aduce o imbunatatire la top K, deci stergem elementul cel mai mare din heap si il adaugam pe cel nou citit

Complexitate : N * logK.

Raspunsul lor: n log k- folosim un maxheap
n * k - 0.5 din punctaj

3.

Cum se poate calcula eficient al *K*-lea cel mai mic număr dintr-o secvență de N numere?

Raspunsul meu: Ne putem folosi de partitionare din algoritmul quicksort.
Ideea este sa nu o ducem sortarea pana la capat si doar pana cand pivotul va fi pe pozitia k.
Astfel vom stii ca toate numerele de pe pozitiile 1...k sunt mai mici decat pivotul, deci pivotul este al k-lea cel mai mic numar.
Raspunsul lor:1p - n (folosind functia de partitionare de la qsort)
0.5p - n log k

4.

Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, dorește să determine toate mulțimile de fortărețe astfel încât chiar dacă una dintre fortărețele dintr-o mulțime ar fi bombardată, celelalte fortărețe din mulțime ar putea să comunice în continuare.

Raspunsul meu: Pentru ca celelate fortarete din multime sa comunice in continuare, fortareata bombardata trebuie sa NU costituie un punct critic in graful format doar din nodurile din aceea multime.
Prin urmare, enuntul este similiar cu gasirea tutoror componentelor biconexe din graf.
Acest lucru poate fi realizat cu algoritmul lui Tarjan
Raspunsul lor: Gasim toate componentele biconexe.

5.

Cum puteți **afișa toate** drumurile distincte între 2 noduri dintr-un graf **orientat aciclic**?

R: Există un număr exponențial de drumuri, deci ar trebui calculate cu backtracking.

6.

Cum puteți **să calculați** numărul total de drumuri distincte între 2 noduri dintr-un graf **orientat aciclic**?

Raspunsul meu: Sortare topologica pe DAG.
Dupa care calculam raspunusul folosind o dinamica.

Raspunsul lor: Sortare topologica apoi dinamică pentru a calcula numarul de drumuri pana la un anumit nod.

Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, se gândește să bombardeze o singură fortăreață, a.î. să întrerupă comunicarea între cât mai multe fortărețe și tabăra lui să câștige. Descrieți un algoritm pentru găsirea fortăreței.

1p - Găsim toate punctele de articulatie și componentele biconexe din jurul lor. Gigel va bombarda punctul de articulație care are în jurul său cele mai multe componente biconexe.

0.5p - Întoarce orice punct de articulație

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mici numere. Secvența este prea mare pentru a fi stocată integral în memorie.

n log k- folosim un maxheap

n * k - 0.5 din punctaj

Se dă un graf neorientat cu *n* noduri cu costuri pozitive pe muchii și *k* surse (k mult mai mic decât n). Pentru fiecare nod din graf se cere drumul de lungime minima față de cea mai apropiata sursă. Descrieți un algoritm eficient pentru această problemă.

Sol1: Se poate adaugă o sursă fictivă cu muchie de cost 0 către toate sursele inițiale și să se facă Dijsktra din aceeastă sursă.

Sol2: Se adaugă toate sursele în heap și continuă cu Dijkstra. Complexitatea este O(M log N). Se puncteaza cu 0.5p o soluție mai ineficientă.

Cum se poate implementa eficient un algoritm de programare dinamică având recurența:

$$dp[n] = \begin{cases} \mathrm{dp}[n-2] - \mathrm{dp}[n-4], & n \geq 4 \\ 2 * n, & n < 4 \end{cases}$$

Exponențiere logaritmică pe matrice

Tribul A este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *X*.
Tribul B este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *Y*.
Fiecare trib are un (singur) loc de unde se poate lua bacul pentru a traversa râul și a ajunge la celălalt trib cu costul *Z*. Aceasta este singura metodă de a ajunge dintr-o parte în alta.
Ce algoritm ați implementa pentru a găsi drumul cu costul cel mai mic dintr-un loc din A (sursă) până la un loc din B (destinație).

pentru BFS maxim, dacă zic BF pentru că au considerat muchii negative also maxim,

dijktra si bf fara explicatie suplimentara, jumatate

Carrefour Orhideea dorește să afle în câte moduri se pot aranja fructele în tăvițe. Se dau *n* tăvițe și *m* fructe (m <= n). Un fruct poate fi pus în orice tăviță. Descrieți un algoritm pentru această problemă.

Se foloseste backtracking pentru a genera toate modalitatile de aranjare pe tavite care respecta restrictiile din enunt.

Cum ați modifica algoritmul lui Dijkstra pentru a calcula al doilea cel mai scurt drum dintr-un graf?

1p - Se păstrează cele mai mici 2 distanțe posibile până la fiecare nod.

0.5p - se scoate pe rând câte o muchie și se rulează Dijkstra

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mici numere. Secvența este prea mare pentru a fi stocată integral în memorie.

n log k- folosim un maxheap

n * k - 0.5 din punctaj

În orașul X, rețeaua de internet este răspândita în toate cartiere, în fiecare cartier fiind amplasate mai multe routere interconectate între ele. Unele routere cedează, iar internetul pică în unul sau mai multe cartiere. Ce algoritm ați folosi pentru identificarea routerelor care duc la astfel de căderi și ce soluție sugerați pentru scăderea numărului de incidente?

Găsim toate punctele de articulație. O soluție ar fi adăugarea de muchii redundante intre cartiere.

Cum se poate calcula eficient al *K*-lea cel mai mic număr dintr-o secvență de N numere?

1p - n (folosind functia de partitionare de la qsort)

0.5p - n log k

Se dă un graf neorientat cu *n* noduri cu costuri pozitive pe muchii și *k* surse (k mult mai mic decât n). Pentru fiecare nod din graf se cere drumul de lungime minima față de cea mai apropiata sursă. Descrieți un algoritm eficient pentru această problemă.

Sol1: Se poate adaugă o sursă fictivă cu muchie de cost 0 către toate sursele inițiale și să se facă Dijsktra din aceeastă sursă.

Sol2: Se adaugă toate sursele în heap și continuă cu Dijkstra. Complexitatea este O(M log N). Se puncteaza cu 0.5p o soluție mai ineficientă.

Cum puteți **afișa toate** drumurile distincte între 2 noduri dintr-un graf **orientat aciclic**?

Există un număr exponențial de drumuri, deci ar trebui calculate cu backtracking

Started on	Wednesday, 3 June 2020, 12:50 PM
State	Finished
Completed on	Wednesday, 3 June 2020, 1:25 PM
Time taken	35 mins 1 sec
Marks	6.00/6.00
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Complete

Mark 1.00 out of 1.00

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mari numere. Secvența este prea mare pentru a fi stocată integral in memorie.

Ma gandesc sa folosesc o coada de prioritati minima, in care voi baga primele K numere. Apoi de la numarul K + 1 pana la N o sa compar minimul din coada cu elementul curent. Daca minimul din coada este mai mare decat numarul curent, scot minimul din coada si adaug noul numar in coada.

De exemplu, pt sirul 1,2,3,0,-1,5 si k = 2 adaug 1,2 in coada dupa la 3 inlocuiesc minimul cu 3, adica voi avea 2 3 in coada, la 0 dau skip, la -1, skip la 5, inlocuiesc 5 cu 2 in coada si obtin 3 5.

Algoritm de tip greedy

n log k- folosim un minheap

n * k - 0.5 din punctaj

Comment:

Question **2**

Complete

Mark 1.00 out of 1.00

Se dă un vector *v* cu *n* intervale de forma [a_i, b_i]. Se cere numărul maxim de intervale disjuncte (care nu se intersectează cu niciun interval). Descrieți un algoritm eficient.

Abordare de tip greedy

Sortez screscator intervalele [ai, bi] dupa timpul de finalizare in n log n. Iau un contor de intervale disjuncte, initializat la 0. Dupa, imi iau start stop la timpul la primul element din vectorul sortat [a0, b0]. Apoi, parcurg vectorul de intervale si daca intervalul curent este continut in intervalul [start, stop] inseamna ca el este redundant si poate fi omis. Altfel, intervalul intervalul curent nu se intersecteaza cu cel anterior si ar trebui sa cresc contor-ul de intervale disjuncte.

Greedy: se sortează apoi se parcurg intervalele de la stânga la dreapta. O(n log n).

Comment:

Question **3**

Complete

Mark 1.00 out of 1.00

Descrieți un algoritm pentru generarea tuturor grafurilor orientate cu *n* noduri care au un ciclu de lungime cel puțin 2.

Cred ca as putea folosi o abordare de tip backtracking in care voi genera toate grafurile posibile si asta imi va lua cam O(2^(n^2)) (ma uit in matricea de adiacenta si iau cele n^2 elemente si le fac pe rand 0 sau 1, mai putin pe cele de pe diagonala principala, care vor ramane 0). Dupa, verific in prima faza daca elementul a[i][j] = a[j][i] in matrice, asta insemnand ca muchia este bidirectionala. Daca nu, aplic un kosaraju/tarjan si pot sa iau toate componentele conexe, dupa care verific daca am mai putin de n componente.

Se folosește backtracking pentru a genera toate submulțimele de muchii care respectă restricțiile din enunț.

Comment:

Question **4**

Complete

Mark 1.00 out of 1.00

Cum puteți **să calculați** numărul total de drumuri distincte între 2 noduri dintr-un graf **orientat aciclic**?

Ma gandesc ca as putea folosi o sortare topologica a nodurilor, fiind DAG. Dupa as aplica o tehnica de programare dinamica, unde as avea un dp[i] = numarul de noduri prin care se poate ajunge din i la destinatie. Ma gandesc, ca as avea doua for-uri (pentru noduri din sortare, pentru vecinii nodului curent), asa ca ma astept la o complexitate n^2. Mai trebuie sa stabilesc dinamica exacta a problemei.

Sortare topologica apoi dinamică pentru a calcula numarul de drumuri pana la un anumit nod.

Comment:

Question **5**

Complete

Mark 1.00 out of 1.00

Se dă un graf orientat cu *x* noduri, având capacitați numere din șirul Fibonacci. Se știe că există *A* surse și *B* noduri destinații. Descrieți un algoritm eficient pentru a transporta o cantitate maximă de flux de la toate sursele la toate destinațiile.

Ma gandesc sa unesc cele A surse si cele B drene intr-un nod sursa si un nod drena. Dupa voi putea aplica algoritmul Edmonds-Karp pentru a gasi cantitatea maxima de flux dintre sursa si destinatie.

Se reunesc toate sursele într-o sursă fictivă S_0 (muchie de la S_0 la S_k de capacitate infinită) și toate destinațiile într-o destinație fictivă T_0 (muchie de la T_k la T_0 de capacitate infinită). S-a obținut o rețea de transport pe care se poate aplica Edmonds-Karp. Graful rezidual se obține ștergând nodurile si muchiile fictive adăugate. Complexitate O(n * m^2) sau O(MaxFlow * m). Se punctează cu 0.5p o soluție mai ineficienta.

Comment:

Question **6**

Complete

Mark 1.00 out of 1.00

Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, dorește să determine toate mulțimile de fortărețe astfel încât chiar dacă una dintre fortărețele dintr-o mulțime ar fi bombardată, celelalte fortărețe din mulțime ar putea să comunice în continuare.

Cred ca voi putea aplica algoritmul lui Tarjan pe aceasta problema, pentru a determina componentele biconexe ale fortaretei. Acest lucru se datoreaza faptului ca daca intr-o componenta biconexa, un nod este eliminat, atunci subgraful asociat componentei biconexe ramane conex. Astfel, daca o fortareata e bombardata, celelalte vor comunica in continuare. Tarjan presupune doar o parcurgere dfs, asa ca ma astept la o complexitate de O(N + M). Daca am fortarete izolate, cred ca nu le voi considera componente biconexe separate, intrucat ele nu comunica cu nimeni, dar nici pe cele cu 2 noduri, intrucat acestea vor deveni izolate. Daca am ceva similar cu un lant, atunci nu cred ca e problema, fiecare mai putin capatul stanga si dreapta fac parte dintr-o comp biconex separaa

Gasim toate componentele biconexe.

Comment:

◄ Examen PA - Subiect II - 03.06.2020

Jump to...

Examen Subiectul 1 - Late submit ►

SUBIECTUL III

Descrieți un algoritm pentru generarea tuturor grafurilor orientate cu *N* noduri pentru care toate gradele de intrare sunt pare.

Se folosește backtracking pentru a genera toate submulțimele de muchii care respectă restricțiile din enunt.

Se dă o matrice *n* x *n*. În *k* celule se află ponei (k mult mai mic decât n). Pentru fiecare celulă din matrice, se dorește să se știe care este distanța minimă către cel mai apropiat ponei. Descrieți un algoritm eficient pentru această problemă.

BFS pe matrice în care se pun în coadă toate celulele cu ponei speciali. Se face BFS pornind cu această coadă și se calculează distanțele. Complexitate O(n^2) unde n e dimensiunea matricei. Se punctează cu 0.5p o soluție mai ineficientă.

Cum ați modifica algoritmul lui Dijkstra pentru a calcula al doilea cel mai scurt drum dintr-un graf?

1p - Se păstrează cele mai mici 2 distanțe posibile până la fiecare nod.

0.5p - se scoate pe rând câte o muchie și se rulează Dijkstra

Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, se gândește să bombardeze o singură fortăreață, a.î. să întrerupă comunicarea între cât mai multe fortărețe și tabăra lui să câștige. Descrieți un algoritm pentru găsirea fortăreței.

1p - Găsim toate punctele de articulatie și componentele biconexe din jurul lor. Gigel va bombarda punctul de articulație care are în jurul său cele mai multe componente biconexe.

0.5p - Întoarce orice punct de articulație

Tribul A este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *X*.
Tribul B este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *Y*.
Fiecare trib are un (singur) loc de unde se poate lua bacul pentru a traversa râul și a ajunge la celălalt trib cu costul *Z*. Aceasta este singura metodă de a ajunge dintr-o parte în alta.
Ce algoritm ați implementa pentru a găsi drumul cu costul cel mai mic dintr-un loc din A (sursă) până la un loc din B (destinație).

pentru BFS maxim, dacă zic BF pentru că au considerat muchii negative also maxim,

dijktra si bf fara explicatie suplimentara, jumatate

Se dă un graf neorientat conex reprezentând rețeaua de socializare Facebook (muchie x-y înseamnă *x* este prieten cu *y*). Gigel a făcut o postare pe Wall, care este văzută de prieteni. Se întreabă dacă postarea poate, ca prin mai multe share-uri, să fie văzută de Botzgorel (care nu este prieten cu Gigel). Descrieți un algoritm eficient.

BFS(Gigel) / DFS(Gigel) și dacă vizităm nodul Botzgorel, atunci răspunsul este DA. Altfel NU.